202 HTTP Status Code in ASP.NET Core Web API

Back to ASP.NET Core Web API Tutorial

202 HTTP Status Code in ASP.NET Core Web API

In this article, I will discuss **How to Return the HTTP Status Code in ASP.NET Core Web API** with Examples. Please read our previous article discussing **How to Return JSP HTTP Status Code** from the ASP.NET Core Web API Controller Action Method with Examples.

202 HTTP Status Code:

The 202 HTTP Status Code is known as "Accepted". It is a success response code indicates that the request has been accepted for processing, but the processing has not been completed. It is only a acknowledgment that the request has been received and is being processed. The purpose of this response is to inform the client that the server has accepted the request but the processing is not yet complete. The client can then continue with other tasks while waiting for the server to finish processing the request. The client can check the status of the request later using other mechanisms, such as polling or webhooks.

Request Accepted: The server has received the request and will process it, but the processing has not be completed yet.

Asynchronous Processing: The server has started the request but will continue processing in the background. It takes that the initial request was valid and has been accepted for processing. The full outcome of the requested action hasn't been determined yet, and the processing is ongoing.

Resource Creation: The server has successfully processed the request and created the resource. The response typically includes a location header pointing to the newly created resource.

Partial Processing: The server has made some progress towards fulfilling the request, but not all the steps have been completed. The exact nature of the partial progress depends on the specific implementation of the server.

Status Monitoring: The server provides a way for the client to check the status of the request later. This could be through a dedicated status endpoint or other mechanisms like webhooks.

How to Return 202 HTTP Status Code in ASP.NET Core Web API?

To return 202 HTTP Status Code in ASP.NET Core Web API, we can use the **Accepted, AcceptedAtAction,** or **AcceptedAtRoute** methods from our controller action methods. These methods are useful for creating responses that indicate the request has been accepted for processing, but the processing has not been completed. The choice between these methods depends on the specific requirements of your application and the resources you want to include in the response. The purpose of each method is to provide a way to inform the client that the request has been accepted and is being processed.

Using Accepted in ASP.NET Core Web API:

The Accepted method returns a 202 Accepted response, which indicates that the request has been accepted for processing but the processing is not complete. The following is an example of how to use the Accepted method in an ASP.NET Core Web API. You can place the code in ControllerBase class.

Syntax:

```
Accepted()
Accepted(object? value)
Accepted(string? uri, object? value)
```

Accepted(): This overload creates a response without a URI. It returns a 202 Accepted status code without any additional content in the body. This overload is useful when the client knows that the request has been accepted without needing additional information about the resource.

Accepted(object? value): This overload includes an optional object in the response body. The object will be serialized to JSON and included in the response. This overload is useful when you want to provide some information about the resource that was created, such as its ID or other relevant details.

Accepted(string? uri, object? value): This overload includes both a URI and an optional object in the response. The URI is a string that specifies the location of the resource that was created. This overload is useful when you want to provide both the location of the resource and some additional information about the resource.

Example to Understand Accepted Method:

Let's create a new controller named ProductsController with an example. Let's add an Empty API Controller with the name **ProductController** and then copy and paste the following code. The following code is self-explanatory. A glance through the comments will provide you with more details.

```
using System.Text.Mvc;
using System.Text.Mvc.ModelBinding;
namespace Demo.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductController : ControllerBase
    {
        [HttpGet("{id}")]
        public IActionResult GetProduct(int id)
        {
            // Simulating a product retrieval
            var product = new { Id = id, Name = "Example Product" };
            return Ok(product);
        }

        [HttpPost]
        public IActionResult CreateProduct(Product product)
        {
            // Simulating asynchronous processing
            var processingId = Guid.NewGuid();
            return Accepted($"/api/status/{processingId}", new { ProcessingId = processingId });
        }

        [HttpPut("{id}")]
        public IActionResult UpdateProduct(int id, Product product)
        {
            // Simulating a product update
            product.Id = id;
            return Accepted($"/api/status/{id}", new { Message = "Update processing", ProductId = id });
        }

        [HttpDelete("{id}")]
        public IActionResult DeleteProduct(int id)
        {
            // Simulating a product deletion
            return Accepted($"/api/status/{id}", new { Message = "Deletion processing", ProductId = id });
        }

        [HttpPatch("{id}")]
        public IActionResult PatchProduct(int id, JsonPatchDocument patchDoc)
        {
            // Simulating a partial update
            var processingId = Guid.NewGuid();
            return Accepted($"/api/status/{processingId}", new { Message = "Partial update processing", ProductId = id });
        }

        [HttpGet("status/{processingId}")]
        public IActionResult GetProcessingStatus(Guid processingId)
        {
            // Simulating status check
            var status = new { ProcessingId = processingId, Status = "In progress", CompletedAt = (DateTime?)null };
            return Ok(status);
        }
    }
}
```

Testing the APIs:

Run the application and test the APIs using Postman. Postman Setup, or Http Files. You can test the APIs using Postman as follows. Please change the port number where your application is running.

GET: Retrieving 202 Status Code with URL:

URL: https://localhost:7139/api/Product/1

Method: GET

Response Body: The Response body will be empty.

POST: Retrieving 202 Status Code with Data:

URL: https://localhost:7139/api/Product/CreateProduct

Method: POST

Response Body: The Response body will be empty.

PUT: Retrieving 202 Status Code with Location URL:

URL: https://localhost:7139/api/Product/UpdateProduct/1

Method: PUT

Response Body: The Response body will be empty.

PATCH: Retrieving 202 Status Code with Location URL and Data:

URL: https://localhost:7139/api/Product/UpdateProduct/1

Method: PATCH

Response Body: The Response body will be empty.

Response Header: The response header will contain the location URI, which you can use to get the current status.

GET: Checking the Job Status with Location URL:

URL: https://localhost:7139/api/status/ffb0f0ce-0ccb-4a0a-a7ff-ad1f7ed1c7ce

Method: GET

Response Body: The Response body will contain the data.

Using AcceptedAtAction in ASP.NET Core Web API:

In ASP.NET Core Web API, the AcceptedAtAction method is a way to return a 202 Accepted HTTP status code along with a specified action and route information. This method is useful when you want to indicate that a request has been accepted for processing and provide a way for the client to check the status of the request later. The following code defines a controller named ProductsController with three actions: GetProduct, CreateProduct, and GetProcessingStatus. The CreateProduct action simulates creating a product and returns a 202 Accepted response with a location header pointing to the GetProcessingStatus action. The GetProcessingStatus action provides a way to check the status of the processing using the processingId parameter.

AcceptedAtAction(string? actionName, object? routeValues, object? value): This overload allows you to specify the action name, route values, and an optional object to be included in the response body. The provided action name is used to generate the URL for the location header, and the route values are used to fill in the parameters of the specified action. Each value for URL parameters are expected to become the route parameter. **AcceptedAtAction(string? actionName, object? routeValues, object? value, string? controller):** This overload allows you to specify the action name, route values, an optional object, and the controller name. The controller name is used to generate the location URL, ensuring that the correct controller is referenced. **AcceptedAtAction(string? actionName, string? controllerName, object? routeValues, object? value, string? fragment):** This overload provides all parameters, including the action name, controller name, route values, an optional object, and a fragment. The fragment is used to specify a fragment in the location URL, which can be useful for navigation within a web page.

Example to Understand AcceptedAtAction in ASP.NET Core Web API:

Let's add the following code in the ProductsController controller. Let's add the code just after the GetProduct() Action Method. The following code is self-explanatory. A glance through the comments have been provided for better understanding.

```
namespace Demo.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductController : ControllerBase
    {
        [HttpGet("{id}")]
        public IActionResult GetProduct(int id)
        {
            // Simulating a product retrieval
            var product = new { Id = id, Name = "Example Product" };
            return Ok(product);
        }

        [HttpPost]
        public IActionResult CreateProduct(Product product)
        {
            // Simulating asynchronous processing
            var processingId = Guid.NewGuid();
            return AcceptedAtAction("GetProcessingStatus", new { processingId = processingId }, new { ProcessingId = processingId });
        }

        [HttpPut("{id}")]
        public IActionResult UpdateProduct(int id, Product product)
        {
            // Simulating a product update
            product.Id = id;
            return AcceptedAtAction("GetProduct", new { id = id }, new { Message = "Update processing", ProductId = id });
        }

        [HttpDelete("{id}")]
        public IActionResult DeleteProduct(int id)
        {
            // Simulating a product deletion
            return AcceptedAtAction("GetProduct", new { id = id }, new { Message = "Deletion processing", ProductId = id });
        }

        [HttpPatch("{id}")]
        public IActionResult PatchProduct(int id, JsonPatchDocument patchDoc)
        {
            // Simulating a partial update
            var processingId = Guid.NewGuid();
            return AcceptedAtAction("GetProcessingStatus", new { processingId = processingId }, new { Message = "Partial update processing", ProductId = id });
        }

        [HttpGet("status/{processingId}")]
        public IActionResult GetProcessingStatus(Guid processingId)
        {
            // Simulating status check
            var status = new { ProcessingId = processingId, Status = "In progress", CompletedAt = (DateTime?)null };
            return Ok(status);
        }
    }
}
```

When should you use the 202 Accepted Status Code in ASP.NET Core Web API?

The 202 Accepted status code is used in ASP.NET Core Web API when the server has accepted a request but has not completed the processing yet. This is useful for scenarios where the processing of the request may take a long time, or where the server needs to perform some asynchronous operations before it can fulfill the request. Here are some specific scenarios where you might use the 202 Accepted status code in ASP.NET Core Web API:

Long-Running Processing: When a request initiates a long-running process, the client should not expect an immediate response. The server can acknowledge the request with a 202 Accepted status code and provide a way for the client to check the status later. The 202 Accepted status code lets the client know that the request has been accepted, and the process will be completed at a later time.

Asynchronous Processing: This is a form of long-running process. It occurs when a request triggers a process in the server's backend that does not depend on the client waiting for the response. The server can return a 202 Accepted status code to indicate that the request has been accepted and will be processed asynchronously. For example, scheduling a task or processing a batch of data.

External Dependencies: When the server depends on external services or resources to process the request, it may take a longer time to complete the processing. The server can return a 202 Accepted status code to indicate that the request has been accepted and will be processed asynchronously. For example, submitting a request to a third-party API or a database that may take some time to respond.

Queued Processing: When the server uses a queue to manage incoming requests, it may not be able to process the request immediately. The server can return a 202 Accepted status code to indicate that the request has been accepted and is queued for processing.

Resource Creation: When the server is creating a new resource, it may take some time to complete the creation process. The server can return a 202 Accepted status code to indicate that the request has been accepted and the resource is being created.

When Should the Accepted or AcceptedAtAction or AcceptedAtRoute?

In ASP.NET Core Web API, Accepted, AcceptedAtAction, and AcceptedAtRoute are used to return a 202 Accepted HTTP status code, but each serves a different purpose and provides different information in the response.

Accepted:

This method is used when the server has accepted the request but does not provide additional details about where the resource will be created or where the resource. It's the simplest form of returning a 202 Accepted response. When a request has been accepted, but the server doesn't need to provide any additional information about the resource, you can use the Accepted method. It's useful for scenarios where the client does not need to know the location of the resource or any further details.

AcceptedAtAction:

When you want to return a 202 Accepted status code, include a content header pointing to a specific action within your controller, and provide route values for the action, you can use the AcceptedAtAction method. This method allows you to specify the action name, route values, and an optional object to be included in the response. This method is useful when you want to provide a URL for the client to check the status of a long-running process. The URL is generated based on the specified action and route values, ensuring that the client can easily navigate to the relevant resource.

AcceptedAtRoute:

When you want to return a 202 Accepted status code, include a content header pointing to a specific route within your application configuration. This is useful when you have named routes in your application and want to provide a URL for the client to check the status of a long-running process. The URL is generated based on the specified route name and route values, ensuring that the client can navigate to the relevant resource.

In ASP.NET Core Web API, Accepted, AcceptedAtAction, and AcceptedAtRoute are used to return a 202 Accepted HTTP status code, but each serves a different purpose and provides different information in the response. The choice between these methods depends on the specific requirements of your application and the resources you want to include in the response. The purpose of each method is to provide a way to inform the client that the request has been accepted and is being processed.

In this post, I will discuss **How to Return the JSP HTTP Status Code in the ASP.NET Core Web API** with Examples. Please read our previous article discussing **How to Return 202 HTTP Status Code in ASP.NET Core Web API** Application with Examples. I hope you enjoy this article on 202 HTTP Status Code in the ASP.NET Core Web API.

Previous post: 202 HTTP Status Code in ASP.NET Core Web API
Next post: 204 HTTP Status Code in ASP.NET Core Web API

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *
Email *
Website

Post Comment

© Dot Net Tutorials | Website Designed by Suresh Pasal